



Product & Engineering Reference Document

Version 1.0 | October 2026 | Confidential | North-East India

RideNow — Product & Engineering Reference Document

Version: 1.0 | **Date:** October 2026 | **Status:** Live Production

North-East India's premier ride-hailing and logistics platform — built for safety, speed, and vernacular inclusivity.

Table of Contents

- 1. [Product Overview](#1-product-overview)
- 2. [Technology Stack](#2-technology-stack)
- 3. [Application Architecture](#3-application-architecture)
- 4. [Database Models & Data Architecture](#4-database-models--data-architecture)
- 5. [Core Engine Systems](#5-core-engine-systems)
- 6. [Feature Catalogue — Customer Side](#6-feature-catalogue--customer-side)
- 7. [Driver Identity, Safety & Payout Verification](#7-driver-identity-safety--payout-verification)
- 8. [Feature Catalogue — Admin Side](#8-feature-catalogue--admin-side)
- 9. [API Surface Map](#9-api-surface-map)
- 10. [Payment Architecture & Wallet System](#10-payment-architecture--wallet-system)
- 11. [Safety System](#11-safety-system)
- 12. [Localisation & i18n System](#12-localisation--i18n-system)
- 13. [Real-time & Infrastructure](#13-real-time--infrastructure)
- 14. [Fare Pricing Matrix](#14-fare-pricing-matrix)
- 15. [Cancellation Policy Matrix](#15-cancellation-policy-matrix)
- 16. [Student Discount Program](#16-student-discount-program)
- 17. [Known Gaps & Future Roadmap](#17-known-gaps--future-roadmap)

1. Product Overview

RideNow is a **full-stack, multi-role, real-time ride-hailing and logistics platform** targeting North-East India. The platform simultaneously serves three distinct user roles: **Customers (Riders)**, **Drivers (Partners/Vendors)**, and **Platform Admins** — each with fully separate dashboards, wallet architectures, and permission surfaces.

Core Value Propositions

Area	What RideNow Delivers
Booking	Instant & scheduled ride booking with 5 vehicle types
Pricing	Upfront guaranteed fare — distance-only, never time-based
Safety	Dual OTP (Pickup + Drop-off), Emergency SOS, live tracking, panic alerts
Payments	Prepaid wallet, UPI/card, cash, outstanding balance recovery
Driver Economy	Earnings ledger, auto settlement, bank withdrawal, commission split
Vernacular	English, Hindi, Assamese, Bengali — full UI translation
Transparency	Itemized fare receipt, real-time dynamic cancellation fee quote

Platform Identity

- **Primary Market:** North-East India (Assam, Meghalaya, etc.)
- **Vehicle Categories:** Bike, Auto, Car, Loading (small cargo), Truck (heavy transport)
- **Languages Supported:** English (en), Hindi (hi), Assamese (as), Bengali (bn)
- **Commission Structure:** Platform takes ~10%, driver earns ~90% of total fare

2. Technology Stack

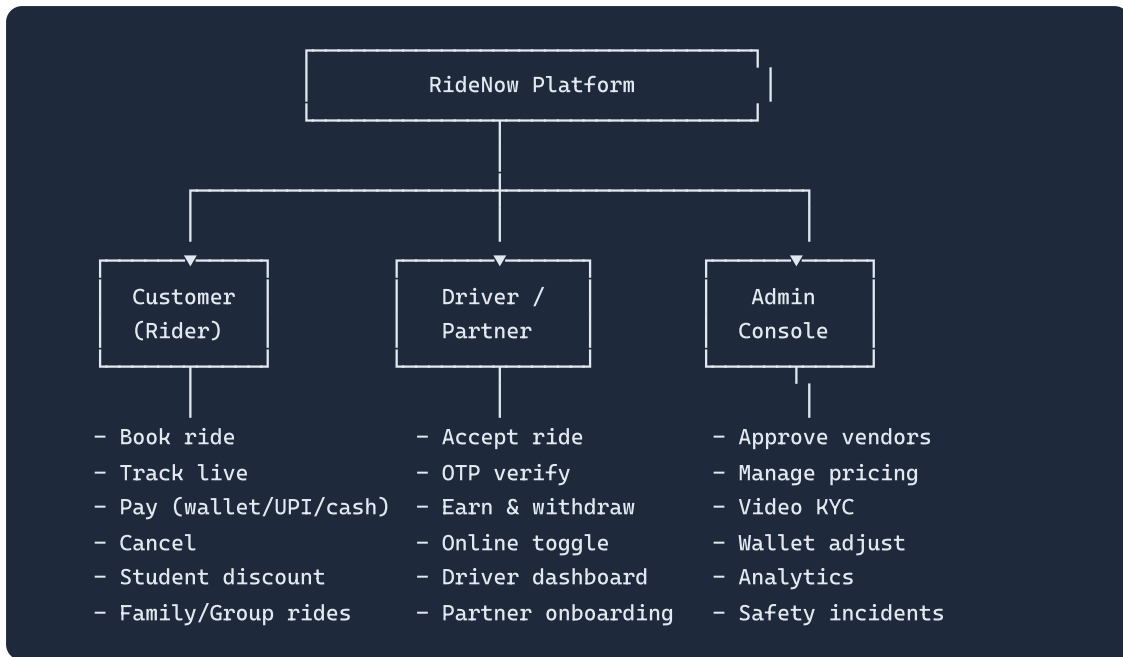
Layer	Technology
Framework	Next.js 16.2.4 (App Router, Turbopack)
Language	TypeScript
UI Library	React 18 + Tailwind CSS + Framer Motion
State Management	Redux Toolkit
Database	MongoDB (Mongoose ODM)
Auth	NextAuth.js v5 (Google OAuth + Mobile OTP)
Real-time	Socket.IO (WebSocket)
Maps	Leaflet.js + OpenStreetMap tiles + Valhalla routing engine
Geocoding / Autocomplete	HERE Maps / HereSearch API (via <code>/api/places</code>)
Payments	Razorpay (UPI, Cards, Netbanking)
Push Notifications	Web Push API (VAPID keys)
Video KYC	Zego Cloud (roomId-based live video sessions)
Media Storage	Cloudinary (vehicle images, documents)
Email / Notifications	Nodemailer (SMTP), WhatsApp API (optional)
OTP Service	Custom OTP service (SMS gateway abstraction)
Build / Deploy	Turbopack production build, static + server-rendered hybrid
Caching / Rate Limiting	Redis

3. Application Architecture

3.1 Folder Structure (Key Modules)

jatri/src/	
app/	← Next.js App Router pages
book/	← Live ride booking interface
ride/[id]/	← Active ride tracking & controls
wallet/	← Customer payment wallet
profile/	← User profile & student settings
bookings/	← Ride history
checkout/	← Fare confirmation + payment flow
fleet/	← Vehicle category showcase
partners/dashboard/	← Main driver dashboard
partner/active-ride/	← Real-time active ride controls
partner/bookings/	← Driver ride history
partner/wallet/	← Driver earnings & withdrawal
partner/onboard/	← 8-step partner onboarding
admin/dashboard/	← Admin console
admin/vendors/[id]/	← Vendor approval/rejection
admin/vehicles/[id]/	← Vehicle approval
track/[token]/	← Public live trip sharing
video-kyc/[roomId]/	← Zego KYC session
api/	← Full REST API (see §9)
booking/	← Core booking lifecycle
partner/	← Driver-facing APIs
wallet/	← Payment & wallet ops
safety/	← Incident & telemetry
admin/	← Admin control APIs
features/	← Feature modules
auth/	← AuthModal, PhoneLinkModal
booking/	← All booking UI components
partner/	← VendorDashboard, DriverWalletCard
maps/	← RouteMap, LiveTrackingMap, DriverLocationMap
rides/	← RideChat, RideSafety, RideCancel, ActiveBanner
admin/	← Charts and dashboards
models/	← MongoDB Mongoose models (see §4)
lib/	← Core engine systems (see §5)
fareEngine.ts	
bookingStateMachine.ts	
driverMatchingEngine.ts	
cancellationEngine.ts	
safetyEngine.ts	
walletLedger.ts	
settlePayment.ts	
serviceArea.ts	
i18n/dictionaries/	
shared/	← Global layout, Navbar, Footer, Hooks
redux/	← Global state slices

3.2 Role-Based Architecture



4. Database Models & Data Architecture

4.1 Core Models

USER MODEL

Unified model for all three roles (user, vendor, admin):

Field Group	Key Fields		
Identity	name , email , password , mobileNumber		
Role	role: "user"	"vendor"	"admin"
Vendor	vendorStatus , vendorOnboardingStep (0-8) , vendorProfileCompleted , vendorRejectionReason		
Video KYC	videoKycStatus , videoKycRoomId , videoKycRejectionReason		
Driver Real-time	isOnline , socketId , currentVehicleType , location (GeoJSON Point) , lastLocationUpdate		
Verification	isEmailVerified , isMobileVerified , otp , otpExpiresAt		
Student Program	isStudent , studentVerification (full sub-document) , studentDetails		
Finance	walletBalance , outstandingAmount		

BOOKING MODEL

The central record linking all actors and the full ride lifecycle:

Field Group	Key Fields
Actors	<code>user</code> , <code>driver</code> , <code>vehicle</code> (ObjectId refs)
Route	<code>pickupAddress</code> , <code>dropAddress</code> , <code>pickupLocation</code> (GeoJSON) , <code>dropLocation</code> (GeoJSON) , <code>stops[]</code>
Fare	<code>fare</code> , <code>adminCommission</code> , <code>partnerAmount</code>
Status	<code>status</code> (BookingStatus) , <code>paymentStatus</code>
OTPs	<code>pickupOtp</code> , <code>pickupOtpExpires</code> , <code>dropOtp</code> , <code>dropOtpExpires</code>
Driver Queue	<code>candidateDrivers[]</code> , <code>currentDriverIndex</code>
Safety	<code>isPanicActive</code> , <code>panicActivatedAt</code> , <code>shareToken</code>
Group Ride	<code>isGroupRide</code> , <code>groupInviteCode</code> , <code>groupMembers[]</code> , <code>splitFarePerPerson</code>
Smart Pickup	<code>isSmartPickup</code> , <code>smartPickupDetails</code>
Scheduling	<code>isScheduled</code> , <code>scheduledPickupTime</code>
Family Ride	<code>isFamilyRide</code> , <code>familyMemberDetails</code>

BOOKING STATUS STATE MACHINE

```
requested
└─> searching_driver
    └─> driver_assigned
        └─> driver_arriving
            └─> driver_arrived
                └─> started / ride_started
                    └─> ride_in_progress
                        └─> completed ← Terminal

Any cancellable state → cancelled ← Terminal
searching_driver      → expired      ← Terminal
                     → no_drivers_available ← Terminal
```

ALL 17 DATABASE MODELS

Model	Purpose
<code>user.model.ts</code>	All user roles (customer, driver, admin)
<code>booking.model.ts</code>	Full ride record and lifecycle
<code>wallet.model.ts</code>	Per-user wallet with balance
<code>wallet-transaction.model.ts</code>	Immutable ledger entries
<code>fareConfig.model.ts</code>	Admin-configurable per-vehicle rate overrides
<code>vehicle.model.ts</code>	Driver's registered vehicle
<code>vehicleDocument.model.ts</code>	KYC documents (RC, license, insurance)
<code>partnerBank.model.ts</code>	Driver's registered bank account
<code>family.model.ts</code>	Family account with linked members
<code>safetyIncident.model.ts</code>	Immutable incident records
<code>studentConfig.model.ts</code>	Admin-configurable student discount parameters
<code>payment.model.ts</code>	Razorpay payment order tracking
<code>withdrawal.model.ts</code>	Driver withdrawal requests
<code>chatMessage.model.ts</code>	In-ride chat messages
<code>push-subscription.model.ts</code>	Web Push subscription endpoints
<code>legal-consent.model.ts</code>	DPDPA/legal consent records
<code>family.model.ts</code>	Family accounts & emergency contacts

5. Core Engine Systems

5.1 Fare Engine (`fareEngine.ts`)

Philosophy: Distance-only guaranteed upfront pricing. Time is informational only — never added to fare. **Fare Calculation Formula:**

```
Total Fare = BaseFare
              + (Distance km × PricePerKm)
              + PlatformFee
              + WaitingCharge (if driver waited > free minutes)
              + 5% GST / Tax
              + SurgeAmount (future)
              - StudentDiscount (10%, max ₹50)
              - PromoDiscount
              + PreviousOutstandingDues (auto-prepended)
```

Waiting Charge Logic:

```
Free Waiting Minutes (3-10 by vehicle) → No charge
After Free Minutes: (actual wait - free minutes) × perMinuteRate = billable
```

Admin Override: Admins can push live `FareConfig` records to MongoDB to override per-vehicle defaults without any code deploy.

5.2 Booking State Machine (`bookingStateMachine.ts`)

Enforces canonical lifecycle transitions with:

- **Race condition protection** via atomic MongoDB `findOneAndUpdate` with status pre-check
- **Idempotency** — same transition returns success without re-applying
- **Role-based permissions:**

Transition Target	Allowed Roles
<code>driver_assigned</code>	driver, system, admin
<code>driver_arrived</code>	driver, system, admin
<code>started</code>	driver, system, admin
<code>completed</code>	driver, system, admin
<code>cancelled</code>	user, driver, system, admin
<code>expired</code>	system, admin

5.3 Driver Matching Engine (`driverMatchingEngine.ts`)

Waterfall queue matching:

1. Geospatial MongoDB `$near` query for online drivers within radius (default 10km)

2. Filters: online status, vehicle type match, location freshness (< 5 min), service area validation
3. Ranked by proximity (closest first)
4. Returns up to 5 prioritized candidates
5. Sequentially notifies drivers — timeout escalation to next if rejected/no response
6. Auto-rematch on full queue exhaustion shows "no drivers available"

5.4 Cancellation Engine (`cancellationEngine.ts`)

Per-vehicle tiered matrix:

Vehicle	En-Route Fee	Driver Arrived Fee
Bike	₹25	₹40
Auto	₹35	₹50
Car	₹45	₹60
Loading	₹60	₹80
Truck	₹80	₹100

Dynamic assessment — policy categories:

- `NO_DRIVER` → ₹0
- `GRACE_PERIOD` → ₹0 (within 2 min of driver accept)
- `DRIVER_DELAY_WAIVED` → ₹0 (driver > 500m AND passenger cites delay)
- `DRIVER_EN_ROUTE` → en-route tier fee
- `DRIVER_ARRIVED` → arrived tier fee
- `DRIVER_CANCELLED` → ₹0 to customer

Settlement split: 70% to driver, 30% to platform **Outstanding flow:** Shortfall →

`outstandingAmount` → auto-recovered on next payment

5.5 Wallet Ledger (`walletLedger.ts`)

Atomic credit/debit with immutable `WalletTransaction` record per event.

Transaction Types: `topup` | `ride_payment` | `refund` | `cancellation_fee` | `cancellation_compensation` | `outstanding_recovery` | `cash_collection` | `platform_commission` | `withdrawal` | `promotional_credit` | `referral_bonus`

5.6 Safety Engine (`safetyEngine.ts`)

- **SOS/Panic:** Creates immutable `SafetyIncident`, sets panic flags, fires push to all emergency contacts
- **Telemetry:** GPS signal drops, unexpected stops, route deviations
- **Incident Types:** `sos`, `harassment`, `route_deviation`, `unsafe_driving`, `vehicle_mismatch`, `suspicious_behavior`, `accident`
- **Severity Levels:** `low`, `medium`, `high`, `critical`
- **Lifecycle:** `active` → `investigating` → `resolved` / `false_alarm`

6. Feature Catalogue — Customer Side

6.1 Authentication

- Google OAuth (NextAuth.js)
- Mobile OTP (6-digit SMS)
- Phone number linking to Google account
- Persistent session

6.2 Ride Booking (`/book`)

- Live fullscreen map (Leaflet + OpenStreetMap)
- Pickup/drop autocomplete (HERE Maps API + geolocation)
- Quick destination chips: Airport, Railway Station, City Center, Metro, Hospital
- Recent destinations (last 10, localStorage)
- **Smart Pickup Spots** — venue-specific designated pickup points
- **Intermediate Stops** — up to 2 waypoints
- Vehicle selection cards with estimated fares
- **Transparent fare receipt** (accordion) — base fare, distance, platform fee, taxes, discounts
- **Instant vs Scheduled Ride** toggle (≥30 min ahead)
- **Family Rider Selector** — book for family member / contact
- **Student Pass banner** — discount applied if verified
- Mobile number verification before confirm
- **Active Ride Protection** — blocks double-booking
- Distance validation (warns if beyond vehicle's max range)
- Translated UI: pickup placeholder, drop placeholder, vehicle labels, CTA buttons

6.3 Checkout (`/checkout`)

Full itemized fare:

- Base Fare, Distance Fare, Platform Fee, GST 5%

- Student Discount (–10%, max ₹50)
- Previous Outstanding Dues (auto-added)
- **Payment Methods:** Wallet | Razorpay UPI/Card | Cash

6.4 Active Ride Tracking (`/ride/[id]`)

- Real-time map with driver's live GPS position
- ETA countdown
- Status progression display
- **Pickup OTP** (customer shows PIN → driver enters → ride starts)
- **Drop-off OTP** (customer shows PIN → driver enters → ride completes)
- In-app audio call (Zego Cloud masked)
- In-app chat (real-time)
- Trip sharing (public link with live tracking, no login needed)
- **Emergency SOS** (one-tap panic protocol)
- **Dynamic cancellation quote modal** (real-time fee, breakdown, outstanding/wallet split)
- Ride Completed View with receipt
- No-Driver / Rematch view

6.5 Customer Wallet (`/wallet`)

- Balance display
- Quick top-up amounts (₹100, ₹500, ₹1000, custom)
- Metric cards: Total Added, Total Spent, Total Transactions
- Outstanding dues banner with "Clear Dues" CTA
- Full transaction passbook (filterable)
- RBI-compliant 256-bit encryption notice

6.6 User Profile (`/profile`)

- Name, email, phone editing
- Phone number linking
- Student Benefits section (verify/re-verify)
- Family Account (add/manage members)
- Safety: trusted emergency contacts
- Account deletion

6.7 Ride History (`/bookings`)

- Past rides with status, fare, pickup/drop
- Filterable by status
- Detailed receipt view per booking

6.8 Group Rides

- Create ride, share invite code
- Multiple users join one booking
- Fare split equally per person

6.9 Legal & Support Pages

Terms, Privacy, Cookies, Cancellation & Refund, Payment Terms, Partner Terms, Safety, Contact, FAQ, Grievance

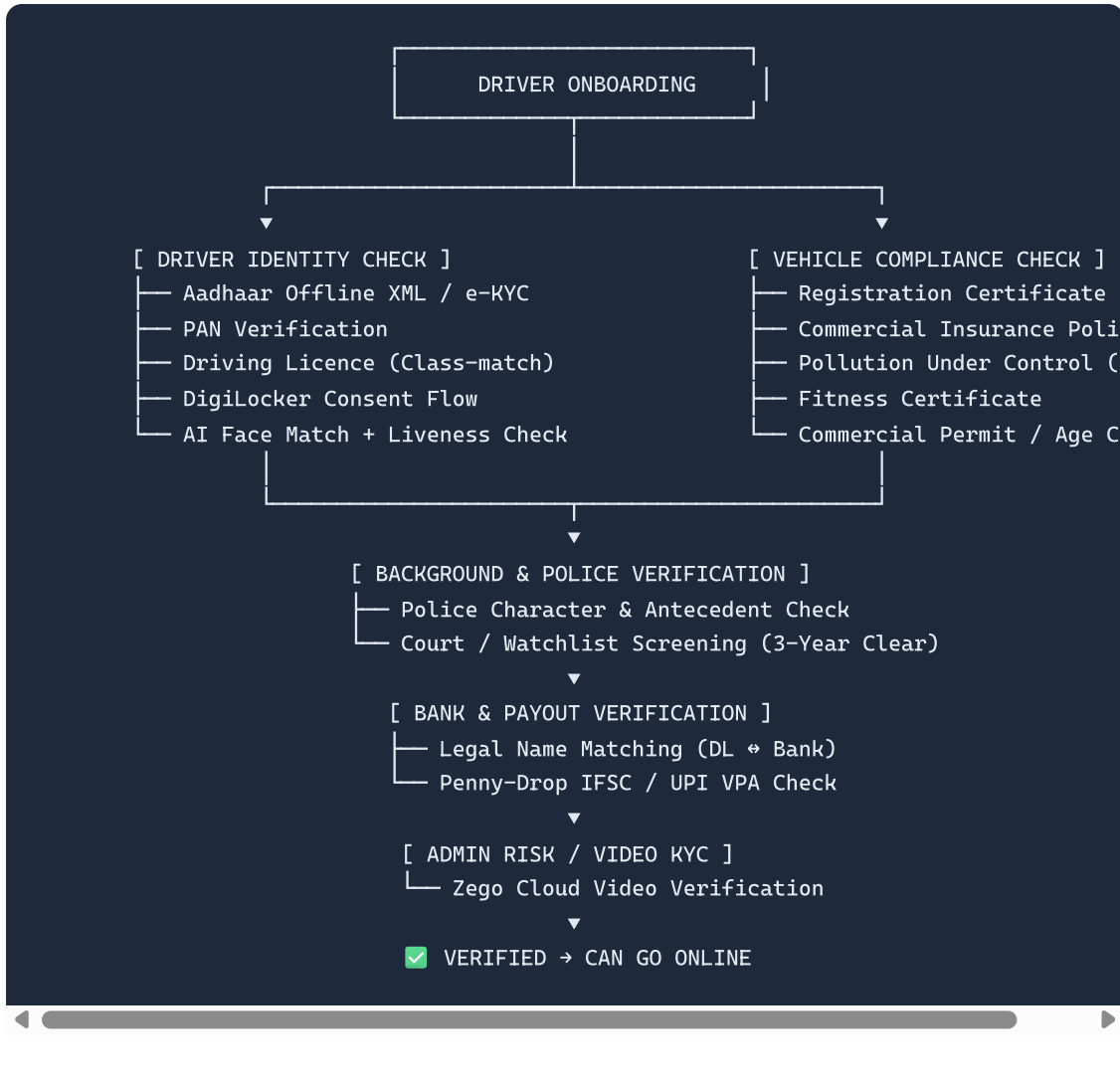
7. Driver Identity, Safety & Payout Verification

Terminology Standard:

- **Driver:** *The individual onboarded, verified, operating the vehicle, and providing transportation.*

- **Rider / Customer:** *The passenger booking and taking the ride.*

The driver and vehicle verification system is the foundational pillar of safety, legal compliance, and fraud prevention for RideNow. Compliant with the **Ministry of Road Transport & Highways (MoRTH) Motor Vehicle Aggregator Guidelines, 2025** and the **Digital Personal Data Protection (DPDP) Rules, 2025**, RideNow implements a strict verification chain: **Driver Identity + Vehicle Compliance + Payout Binding**.



7.1 Digital Identity Verification

Digital identity verification establishes the driver's real-world legal identity before the driver can accept any booking.

- **Aadhaar Offline e-KYC / XML:**

Drivers provide UIDAI's digitally signed Offline paperless e-KYC XML (with Share Code).

Privacy Guard: Allows demographic and photo verification without storing the unmasked 12-digit Aadhaar number in RideNow databases (storing only masked `XXXX-XXXX-1234` and UIDAI verification hash).

- **PAN Verification:**

Validates permanent account number against NSDL/Income Tax database for identity, taxation (TDS deduction under 194O), and payout KYC records.

- **Driving Licence (DL) Verification:**

Validates licence number against the Sarathi/Parivahan national database. Verifies:

- Driver full legal name and date of birth

- Validity status (active, non-suspended)
- Permitted vehicle class (e.g., LMV-TR / Transport endorsement for commercial four-wheelers, MCWG for bikes, 3-Wheeler Cab for autos).

- **DigiLocker Verification:**

Enables consent-driven automated retrieval and verification of government-issued credentials (DL, RC, Insurance) directly from source authorities.

- **MoRTH 2025 Aggregator Compliance:**

Ensures statutory compliance with Section 6 of the 2025 Guidelines requiring aggregators to confirm driver identity proof and class-appropriate driving licences prior to passenger onboarding.

7.2 AI Face Match & Liveness Verification

Prevents driver account sharing, profile fraud, and driver impersonation.

- **Selfie ↔ ID/DL Face Match:**

Extracts driver facial embeddings from the live onboarding selfie and matches them against the photo extracted from the verified DL and Aadhaar e-KYC using biometric comparison algorithms.

- **Active & Passive Liveness Detection:**

Ensures physical human presence during selfie capture (detects printed photos, digital screens, deepfakes, and masks via eye blink, head turn, and texture analysis).

- **Periodic Pre-Shift Re-verification:**

Drivers are prompted for a mandatory live selfie check at randomized intervals and prior to beginning new shifts.

- **Risk & Anomaly Triggers:**

Automatic lock and administrative review upon:

- Login from unrecognized device hardware or unusual IP geolocation
- Multiple failed face matches
- Sudden change in device biometric signature

```
Driver Open App → Live Selfie → Liveness Verification → Face Match
├── ✓ Pass (Score ≥ 85%) → Allowed to Go Online
└── ✗ Fail → Temporarily Blocked → Admin Manual Review / Video KYC
```

7.3 Criminal & Background Verification

Driver safety layer adhering to regulatory aggregator standards rather than ad-hoc internet searches.

- **Police Character & Antecedent Verification:**

Maintains official records of state police character verification certificates (PCC) or antecedent verification reports.

- **3-Year Conviction Clearance:**

Strictly complies with the 2025 MV Aggregator Guidelines requirement that onboarded drivers must not have been convicted within the preceding three years of offenses related to rash/negligent driving, DUI, assault, or moral turpitude.

- **Court & Watchlist Screening:**

Screening through authorized verification partners across e-Courts and regulatory watchlists.

- **Audit Logging:**

Stores immutable verification certificate references, issuance dates, renewal validity periods, and issuing authority stamps.

7.4 Address Verification

Establishes driver traceability, home domicile, and regulatory emergency-contact records.

- **Permanent & Current Residential Address:** Captured separately to support migrant and urban commercial drivers.
- **PIN-Code Geo-Validation:** Cross-referenced with active RideNow operating municipal clusters.
- **Address Document Proof:** Validated via Aadhaar e-KYC, Voter ID, or utility documents.
- **Location Consistency Checks:** Periodic matching of driver shift start locations against declared domicile to detect anomalous accounts.
- **Emergency Contact Information:** Compulsory storage of two verified family/next-of-kin contacts with phone numbers for safety escalation.

7.5 Bank Account & UPI Verification

Directly binds verified driver identity to the payout destination to eliminate money-laundering and fraudulent withdrawals.

- **Account Ownership & Name Matching:**

Account holder name retrieved via bank Penny-Drop verification (IMPS ₹1 test transfer) or UPI VPA lookup must match the legal name verified on the Driving Licence and PAN with high fuzzy-match confidence ($\geq 80\%$).

- **IFSC Validation:** Real-time bank branch code validation via RBI database.
- **Payout Protection Rule:**

Zero arbitrary payouts: Payouts are restricted exclusively to the verified bank account. Any request to update bank or UPI details immediately freezes withdrawals for 48 hours and requires re-verification.

- **MoRTH 2025 Aggregator Compliance:** Adheres to the statutory mandate requiring aggregators to settle driver earnings strictly into accounts held in the driver's own legal name.

7.6 Vehicle Verification (Comprehensive Vehicle Compliance)

A driver can be genuine, but the vehicle itself may be unregistered, uninsured, or unsafe. RideNow mandates rigorous vehicle-level compliance before any vehicle is attached to a driver.

- **Registration Certificate (RC) Verification:**

Verified via Parivahan / Vahan API. Confirms registration number, engine number, chassis number, owner name, vehicle class, and commercial vs. private registration.

- **Vehicle Class & Seating Capacity:**

Matches vehicle specifications to RideNow fleet tiers (**bike** , **auto** , **car** , **loading** , **truck**).

- **Commercial Insurance Validity:**

Ensures active Third-Party / Comprehensive Commercial Vehicle Insurance policy. Automated expiration tracking alerts the driver 30, 15, and 3 days before policy lapse.

- **Pollution Under Control (PUC) Certificate:**

Validates PUC emission certificate number and valid-until date against national emission registries.

- **Fitness Certificate:**

Mandatory commercial transport fitness validation ensuring the vehicle is roadworthy as certified by the Regional Transport Office (RTO).

- **Commercial Permit Verification:**

Validates Contract Carriage, All-India Tourist Permit, or State Goods Carriage permit as applicable to the vehicle category.

- **Challan & Dues Screening:**

Checks for excessive unpaid traffic violations or pending impound notices.

- **Vehicle Age & Green Norms:**

Enforces municipal age caps (e.g., maximum 8–10 years from registration) to ensure fleet quality.

- **Physical Photo Inspection:**

Mandatory uploads of 4-angle vehicle photos (front with visible number plate, side profile, rear, and clean interior cabin) verified by admin reviewers.

7.7 Driver Verification Status Engine

Rather than relying on a simplistic binary `isVerified: true/false`, RideNow implements a **multi-dimensional verification status engine**.

```
export interface IDriverVerificationStatus {
  identity: "not_submitted" | "pending" | "verified" | "rejected" | "expired";
  drivingLicense: "not_submitted" | "pending" | "verified" | "rejected" | "expired";
  face: "not_submitted" | "pending" | "verified" | "rejected" | "expired";
  background: "not_submitted" | "pending" | "verified" | "rejected" | "expired";
  address: "not_submitted" | "pending" | "verified" | "rejected" | "expired";
  bank: "not_submitted" | "pending" | "verified" | "rejected" | "expired";
  vehicle: "not_submitted" | "pending" | "verified" | "rejected" | "expired";
}
```

DERIVED "GO ONLINE" ELIGIBILITY FORMULA:

```
const canGoOnline =
  status.identity === "verified" &&
  status.drivingLicense === "verified" &&
  status.face === "verified" &&
  status.background === "verified" &&
  status.bank === "verified" &&
  status.vehicle === "verified" &&
  !user.isVendorBlocked;
```

Why this architecture is critical:

- Enables granular document expiry tracking (e.g., vehicle insurance expires while driver licence remains valid).
 - Automatically sets `canGoOnline = false` the moment any individual pillar enters `"expired"` or `"rejected"`.
 - Supports partial onboarding progression across multiple sessions.
 - Facilitates fast re-verification without restarting the entire onboarding process.
-

7.8 Verification Prioritization Matrix

Pillar	Priority	Justification
Mobile OTP	Critical	Confirms account ownership and primary device channel
Driving Licence	Critical	Fundamental legal prerequisite to operate motor vehicles
Identity (Aadhaar/PAN)	Critical	Establishes enforceable civil identity and tax compliance
Vehicle RC	Critical	Legally ties the physical vehicle to the driver profile
Commercial Insurance	Critical	Passenger and third-party financial liability protection
Bank Account Match	Critical	Statutory compliance and fraud-proof payout pipeline
Face Match & Liveness	High	Eradicates account renting, sharing, and driver impersonation
PUC & Fitness Certificate	High	Environmental compliance and roadworthiness certification
Police / Antecedent Check	Critical (Prod)	Direct safety requirement under MoRTH Aggregator Guidelines 2025
Commercial Permit	High	Transport compliance for taxi / loading commercial classes
Address Verification	Medium	Driver traceability and emergency handling
Physical House Check	Low	High operational cost; reserved for high-risk / disputed cases

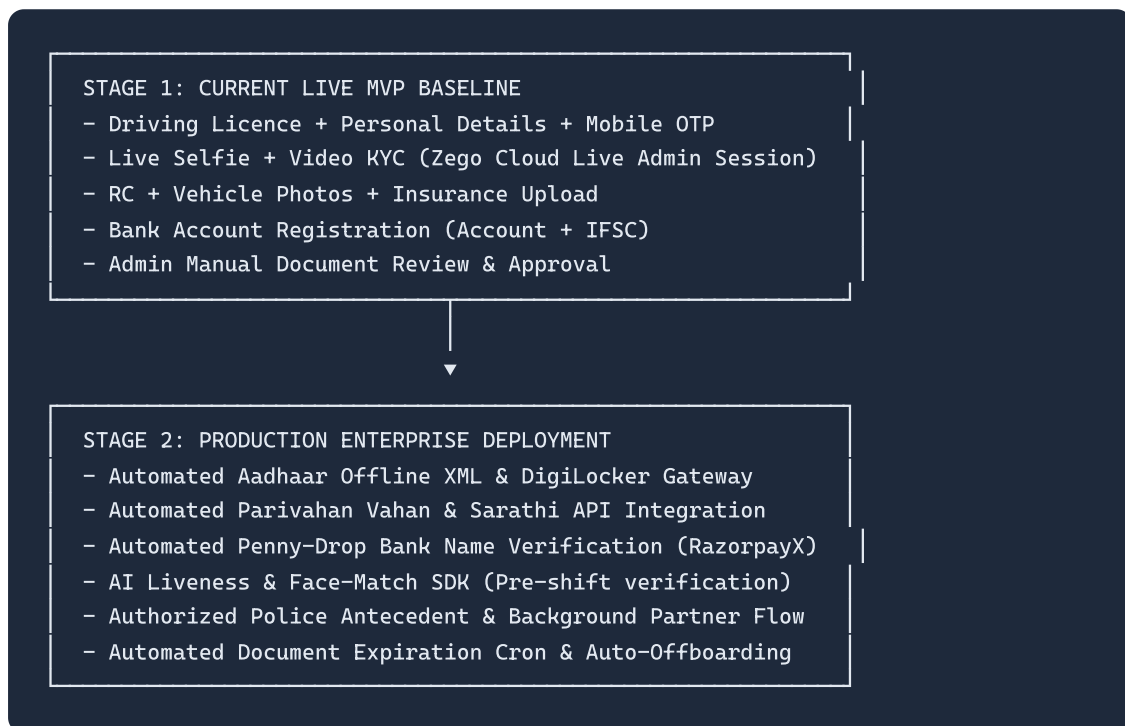
7.9 Data Privacy & DPDP Rules 2025 Compliance

RideNow's data architecture is constructed around the **Digital Personal Data Protection Act (DPDP), 2025**:

- **Purpose Limitation:** Driver data (biometrics, documents, financials) is utilized exclusively for verification and payout processing.
- **Zero Raw Aadhaar Storage:** Aadhaar numbers are never stored in plaintext. Only masked formats and UIDAI digital transaction reference tokens are retained.

- **Encryption at Rest & Transit:** All document URLs, banking IFSCs, and personally identifiable information (PII) are encrypted via AES-256 and transmitted over TLS 1.3.
- **Granular Consent Management:** Consent is explicitly recorded with timestamp, purpose ID, and IP address in the `LegalConsent` collection prior to document upload.
- **Retention & Deletion Policy:** Driver PII can be purged or anonymized upon validated account closure, maintaining only statutory transaction records for the legally prescribed tax audit period.

7.10 Two-Stage Implementation Roadmap



7.11 Driver Operations & App Features

DRIVER DASHBOARD (`/PARTNERS/DASHBOARD`)

- **Header:** Driver avatar, verified badge, and quick links to incoming requests and ride history.
- **Shift Performance Bar (4 Metrics):**
 - Today's Earnings (₹)
 - Trips Completed
 - Online Shift Duration (hrs)
 - Driver Rating & Acceptance Rate (%)
- **Status Command Card:** Online/Offline toggle with real-time GPS permission checks and continuous location telemetry.

- **Recent Trips & Drop-Off Ledger:** Detailed per-trip passenger drop-off times, fares, and net driver share.
- **Earnings Trend Chart:** Daily and weekly performance visualisations.

INCOMING RIDE RADAR MODAL

Socket.IO `new-booking` event triggers an urgent, audio-enabled modal:

- 20-second countdown timer
- Pickup & drop-off locations with estimated travel distance and duration
- **Guaranteed Net Fare** displayed prominently
- Instant Accept / Reject actions

DRIVER EARNINGS LEDGER (`/PARTNER/WALLET`)

- **Ledger Model:** Earnings and settlement tracking only — **no "Add Money / Top Up" buttons.**
- **Financial Balance Metrics:** Available to Withdraw, Pending In-Flight Earnings, Platform Dues.
- **Withdrawal Action:** Instant withdrawal request directly to verified bank account via IMPS/NEFT.
- **Auto-Split Ledger:** Automatic real-time credit of 90% fare to driver and 10% platform commission deduction upon drop-off OTP validation.

ACTIVE RIDE CONTROLS (`/PARTNER/ACTIVE-RIDE`)

- "I'm Arriving" & "I've Arrived" status updates with passenger notifications.
- **Pickup OTP Verification Gate:** 4-digit code provided by passenger required to start trip.
- **Drop-off OTP Verification Gate:** 4-digit code provided by passenger required to end trip and trigger financial settlement.
- Live in-app navigation and real-time chat with passenger.

8. Feature Catalogue — Admin Side

8.1 Admin Dashboard

- Platform earnings overview
- Total rides, drivers, customers
- Booking status breakdown (pie chart)
- Earnings trend chart

8.2 Vendor Management

- View full vendor profile + documents
- Approve / Reject applications with reason

- Trigger Video KYC (Zego)
- Block / unblock vendors

8.3 Vehicle Management

- Approve / Reject vehicle registrations
- View vehicle photos and documents

8.4 Pricing Management

- Override per-vehicle fare rates live (no code deploy)

8.5 Student Program Config

- Discount %, max cap, min fare, eligible vehicles, stacking toggle

8.6 Wallet Adjustments

- Credit or debit any user's wallet with admin note + audit trail

8.7 Withdrawal Management

- Review and approve driver withdrawal requests
- Process IMPS/NEFT payouts

8.8 Safety Incident Management

- View all incidents with severity
 - Update incident lifecycle status
-

9. API Surface Map

Booking APIs (24 endpoints)

Endpoint	Method	Purpose
/api/booking/create	POST	Create booking + driver matching
/api/booking/[id]	GET	Fetch booking details
/api/booking/[id]/status	GET	Status polling
/api/booking/[id]/accept	POST	Driver accepts
/api/booking/[id]/reject	POST	Driver rejects
/api/booking/[id]/arriving	POST	Driver en-route
/api/booking/[id]/arrived	POST	Driver at pickup
/api/booking/[id]/start	POST	Pickup OTP verified → ride starts
/api/booking/[id]/complete	POST	Drop OTP verified → ride completes
/api/booking/[id]/cancel	GET	Real-time dynamic fee quote
/api/booking/[id]/cancel	POST	Confirm cancel + settlement
/api/booking/[id]/rematch	POST	Re-match to next candidate
/api/booking/[id]/timeout	POST	Driver timeout → escalate
/api/booking/[id]/expire	POST	Expire unmatched booking
/api/booking/[id]/panic	POST	SOS panic protocol
/api/booking/[id]/safety-checkin	POST	Periodic telemetry check
/api/booking/[id]/reschedule	POST	Reschedule
/api/booking/[id]/confirm-payment	POST	Payment received
/api/booking/[id]/group/invite	POST	Add group member
/api/booking/my-active	GET	Customer's active booking
/api/booking/group/join	POST	Join group ride
/api/booking/scheduled/dispatch	POST	Dispatch scheduled rides

Partner APIs (18 endpoints)

`/api/partner/status` (GET/PATCH), `/api/partner/bookings` + active + pending + counts, pickup/drop OTP send+verify (4), earnings, shift-summary, wallet + withdraw, vehicle + pricing, documents, bank, video-kyc request + start

Wallet APIs

`/api/wallet` (GET), `/api/wallet/pay` (POST), `/api/wallet/topup/create` (POST), `/api/wallet/topup/verify` (POST)

Payment APIs

`/api/payment/create`, `/api/payment/verify`, `/api/payment/webhook`

Auth APIs

`/api/auth/[ ... nextauth ]`, phone send-otp, phone verify-otp, phone link, register

Admin APIs

dashboard, earnings, pricing, student-config, vehicles approval, vendors approval, video-kyc management, wallet/adjust, withdrawals, safety/incidents

Utility APIs

`/api/places` (autocomplete + details), `/api/places/smart-pickups`, `/api/route` (Valhalla), `/api/vehicles/nearby`, `/api/vehicles/pricing`, `/api/me`, `/api/chat/send` + get-all, `/api/track/[token]` (public), `/api/zego/token`, `/api/user/bookings` + family + verify-student + delete-account, `/api/safety/incident` + telemetry, `/api/notifications/subscribe` + unsubscribe + vapid-key, `/api/legal/consent`

10. Payment Architecture & Wallet System

Customer Wallet Flow

```

Customer (UPI/Card/Cash)
  ↓
Razorpay Gateway
  ↓
Razorpay Webhook → /api/wallet/topup/verify
  ↓
Customer Wallet balance increases
+ WalletTransaction record (immutable)
  ↓
Ride Booking → Wallet deducted at checkout
+ Outstanding Dues auto-prepended if any
  
```

Driver Earnings Flow

```

Drop OTP Verified → Ride Completed
  ↓
settlePayment.ts
  ↓
Fare Split:
90% → Driver Wallet (partnerAmount)
10% → Platform (adminCommission)
  ↓
WalletTransaction records created for both
  ↓
Driver requests withdrawal
  ↓
Admin approves → IMPS/NEFT payout
  
```

Outstanding Dues Recovery

```

Cancellation fee applied
  ↓
Wallet sufficient → deduct immediately
Wallet insufficient → remainder → user.outstandingAmount
  ↓
On next payment/top-up → outstandingAmount auto-prepended
(Customer cannot bypass – always recovered)
  
```

11. Safety System

Dual OTP Flow

```
Driver Arrives at Pickup
→ Pickup OTP shown on customer screen (4-digit, time-limited)
→ Customer tells driver the code
→ Driver enters OTP → ride STARTS

At Destination
→ Drop-off OTP shown on customer screen (4-digit, time-limited)
→ Customer tells driver the code
→ Driver enters OTP → ride COMPLETES + payment settles
```

Emergency SOS

- One-tap button on active ride screen
- Notifies ALL trusted emergency contacts (push + message)
- Sets `isPanicActive = true` on booking
- Creates immutable `SafetyIncident` record
- Admin alerted

Trip Sharing

- Public `shareToken` link → live driver location visible
- No login needed for viewer
- Personal details masked

Telemetry Monitoring

- Periodic check-ins during active ride
 - Flags: GPS loss, unexpected stops, route deviations
 - Logged as `SafetyIncident` when triggered
-

12. Localisation & i18n System

Supported Languages

Code	Language
en	English
hi	Hindi
as	Assamese
bn	Bengali

Architecture

- **Centralized dictionaries:** `src/lib/i18n/dictionaries/{en,hi,as,bn}.json`
- **Global Context:** `LanguageContext` + `useTranslation()` hook
- **Usage:** `t("namespace.key", "Fallback text")`
- Global re-render on language switch

Translation Namespaces

nav | hero | booking | checkout | tracking | cancellation | wallet | student |
profile | driver | fleet | auth | footer | alerts | common

Fully Translated Components

- Navigation, Hero Section, Booking form (inputs, chips, CTAs)
- Vehicle cards (labels, descriptions), Fare receipt
- Checkout flow, Customer wallet, Driver dashboard
- Active ride tracking, Cancellation flow, Auth modal
- Profile sections, Footer, Error messages

13. Real-time & Infrastructure

Socket.IO Events

Direction	Event	Purpose
Server → Driver	new-booking	New ride request
Server → Driver	new-scheduled-booking	Scheduled ride alert
Server → Client	booking-updated	Status change
Server → Customer	driver-location-update	GPS position
Driver → Server	identity	Associate socket with user
Driver → Server	update-location	Push live GPS
Client → Server	join-ride-room	Subscribe to booking updates

Web Push Notifications

Used for: new ride requests (driver), SOS alerts (emergency contacts), cancellation alerts, payment confirmations

Routing (Valhalla)

Self-hosted routing engine for: distance calculation, ETA, route polylines. Haversine fallback on failure.

Geocoding (HERE Maps)

Autocomplete with bounding box filtering relative to pickup + vehicle max range.

Idempotency (Redis)

Redis-backed idempotency keys on booking create. Prevents duplicate bookings from double-taps.

Rate Limiting (Redis)

Protects OTP send and booking create endpoints.

14. Fare Pricing Matrix

Default Vehicle Rates

Vehicle	Base Fare	Per Km	Platform Fee	Wait/min	Free Wait	Max Range
Bike	₹30	₹9	₹15	₹1.50	3 min	15 km
Auto	₹45	₹13	₹15	₹2.00	3 min	30 km
Car	₹75	₹18	₹15	₹2.50	4 min	100 km
Loading	₹110	₹22	₹20	₹3.50	5 min	150 km
Truck	₹160	₹28	₹30	₹5.00	10 min	500 km

All fares + 5% GST/Tax. Admin can override any rate live.

Sample Car Fares (no discounts)

Distance	Approx Total
5 km	~₹157
10 km	~₹199
20 km	~₹283
30 km	~₹367

15. Cancellation Policy Matrix

By Scenario

Scenario	Fee	Category
No driver found	₹0	NO_DRIVER
Within 2 min of driver accept	₹0	GRACE_PERIOD
Driver > 500m + passenger cites delay	₹0	DRIVER_DELAY_WAIVED
Driver en-route, after grace	Per-vehicle en-route fee	DRIVER_EN_ROUTE
Driver has arrived at pickup	Per-vehicle arrived fee	DRIVER_ARRIVED
Driver cancelled	₹0 to customer	DRIVER_CANCELLED

Tiered Fees by Vehicle

Vehicle	En-Route	Arrived
Bike	₹25	₹40
Auto	₹35	₹50
Car	₹45	₹60
Loading	₹60	₹80
Truck	₹80	₹100

Settlement Split (when fee applied)

- **70%** → **Driver** (cancellation compensation)
- **30%** → **Platform**

16. Student Discount Program

Design: Annual 12-Month Verified Benefit

Apply → Verify → Approved
→ 10% discount on eligible rides for 12 months
→ After expiry: must re-verify to continue

Rules

Parameter	Default Value
Discount	10% of ride fare
Maximum Per Ride	₹50 (capped)
Minimum Ride Fare	₹100
Validity	12 months
Stackable with Promo	✗ No
Eligible	Standard rides (not premium/luxury)

Verification Methods

- 1. **Institutional Email** — `.edu` or `.ac.in` OTP
- 2. **Student ID Card** — Photo upload + admin review
- 3. **Enrollment / Bonafide Certificate** — Document + admin review

Sample Savings

Ride Fare	Discount	Customer Pays
₹100	₹10	₹90
₹300	₹30	₹270
₹500	₹50 (cap)	₹450
₹1,000	₹50 (cap)	₹950

All parameters configurable by admin at `/api/admin/student-config`.

17. Known Gaps & Future Roadmap

17.1 Currently Live and Working

Feature	Status
Full booking lifecycle (Instant + Scheduled)	✓ Live
5 vehicle categories with pricing	✓ Live
Dual OTP start/end verification	✓ Live
Real-time GPS tracking	✓ Live
Customer wallet (top-up, deduct, history)	✓ Live
Driver earnings ledger (auto-credit, withdraw)	✓ Live
Dynamic cancellation fee + real-time quote	✓ Live
Outstanding dues auto-recovery	✓ Live
Student discount program (12-month verified)	✓ Live
Family rides	✓ Live
Group rides (fare split)	✓ Live
Scheduled rides	✓ Live
Smart pickup spots	✓ Live
Intermediate stops (waypoints)	✓ Live
Emergency SOS + panic protocol	✓ Live
Safety telemetry monitoring	✓ Live
Trip sharing (masked public link)	✓ Live
In-app chat (driver ↔ customer)	✓ Live
In-app audio/video calls (Zego)	✓ Live
Video KYC for drivers	✓ Live
Partner onboarding (8-step)	✓ Live
Admin dashboard	✓ Live
Web push notifications	✓ Live
4-language i18n	✓ Live

Razorpay payment integration	✓ Live
Idempotency (no duplicate bookings)	✓ Live

17.2 Recommended Future Improvements

● HIGH PRIORITY (FIX FIRST)

Area	Gap	Recommended Fix
Language persistence	Language resets on page refresh	Persist to <code>localStorage</code> , optionally sync to user profile
Language dropdown	Overflows viewport on narrow/mobile screens	Viewport-aware positioning with <code>IntersectionObserver</code> or CSS containment
Ratings system	No post-ride driver/passenger ratings	Post-ride rating modal: 1–5 stars + optional comment → driver score updates
Split payment	Wallet + UPI for difference not supported	Allow partial wallet deduction + Razorpay for remainder in one transaction
Surge pricing	No dynamic demand multiplier	Geofence-based demand/supply ratio → 1.0x–2.5x surge multiplier on fare

● MEDIUM PRIORITY (NEXT PHASE)

Area	Gap	Recommended Fix
Promo codes	Referenced in codebase but not built	Full promo code CRUD, per-user redemption, usage limits, expiry, abuse detection
Referral program	Wallet has referral credit type but no referral system	Referral link generation, tracking, automatic credit on first ride
Driver incentives	No bonus/streak system for drivers	Daily streak bonuses (e.g. 10 trips = ₹100), peak hour multipliers
Surge UI	No surge indicator on booking screen	Surge badge on vehicle cards when multiplier > 1.0x
Admin analytics	Basic charts only	Ride heatmaps, driver utilization, cohort retention, city-level breakdown
Scheduled ride UX	Dispatch works but no pre-assignment	T-30 min notification to driver; pre-match driver to scheduled ride
No-driver UX	Basic "no drivers available" screen	Suggest alternative vehicle types, widen radius retry, notify when driver available
Driver cancellations	Driver cancel = ₹0 to customer, but no driver penalty system	Driver penalty points, suspension after N cancellations
Waiting charges UI	Waiting charges are calculated but may not be shown clearly at checkout	Show waiting charge line item prominently on final receipt

● LONG-TERM / GROWTH

Area	Recommendation
Native Mobile Apps	React Native for Android + iOS (driver app especially — background GPS, native notifications)
Intercity Rides	Long-distance booking with multiple vehicle handoffs and rest stops
Corporate / B2B	Corporate accounts with monthly billing, consolidated receipts, department codes
Logistics Open API	REST API for businesses to book loading/truck rides programmatically
Multi-city Expansion	Per-city service area management with city-level pricing tiers
Toll Integration	Factor toll charges on specific highway routes into fare
Per-ride Insurance	Micro-insurance product bundled per ride (₹5–₹15 optional add-on)
Fraud Detection	ML-based anomaly detection: fake trips, GPS spoofing, fake cancellations
Offline Mode	Driver app works offline, syncs queue when reconnected
EV Fleet Support	EV-specific categories with charging station awareness
Accessibility	Screen reader support, high-contrast mode, large text mode
Fare Appeal	Customer can dispute a fare with evidence; admin arbitration flow

Appendix: Key File Reference

File	Purpose
<code>src/lib/fareEngine.ts</code>	Pricing calculation engine
<code>src/lib/bookingStateMachine.ts</code>	State transition guard with role permissions
<code>src/lib/driverMatchingEngine.ts</code>	Driver waterfall matching with escalation
<code>src/lib/cancellationEngine.ts</code>	Dynamic cancellation fee + assessment
<code>src/lib/safetyEngine.ts</code>	SOS, telemetry, safety incidents
<code>src/lib/walletLedger.ts</code>	Atomic wallet credit/debit operations
<code>src/lib/settlePayment.ts</code>	Post-ride fare settlement (driver + platform split)
<code>src/lib/serviceArea.ts</code>	Geofencing, service area validation, recent destinations
<code>src/models/booking.model.ts</code>	Core booking schema with all lifecycle fields
<code>src/models/user.model.ts</code>	Unified user + driver + vendor schema
<code>src/app/book/page.tsx</code>	Customer booking interface (1,600+ lines)
<code>src/app/ride/[id]/page.tsx</code>	Active ride tracking UI with OTPs, SOS, cancel
<code>src/features/partner/components/VendorDashboard.tsx</code>	Full driver dashboard
<code>src/lib/i18n/dictionaries/en.json</code>	Master English translation dictionary

RideNow Product & Engineering Reference | October 2026 North-East India's premier ride-hailing and logistics platform

RideNow Product & Engineering Reference | October 2026 | North-East India's premier ride-hailing and logistics platform